



Politechnika
Wrocławska

ML-Based HTTP Threat Detection



Autor: Kacper Zawadzki

Agenda

1. Cel i zakres projektu
2. Problem rozpoznawania inputu
3. Architektura i pipeline rozwiązania
4. TF-IDF - czym jest i jak działa?
5. Charakterystyka wykorzystanych algorytmów klasyfikujących
6. Porównanie klasyfikatorów (walidacja krzyżowa) - **eksperyment 1**
7. Wpływ oversamplingu na skuteczność modeli - **eksperyment 2**
8. Analiza wyników i wnioski
9. Podsumowanie projektu i świadomość ograniczeń

Cel i zakres projektu

1. **Główny cel:** Zbadanie skuteczności algorytmów uczenia maszynowego w wykrywaniu anomalii i ataków w ruchu HTTP (m.in. SQL Injection, XSS, Path Traversal).
2. **Zakres danych:** Wykorzystanie zbioru *CSIC 2010* do analizy surowych żądań (URL oraz zawartość POST).
3. **Inżynieria cech (Feature Engineering):** Zastosowanie wektoryzacji *TF-IDF* w oparciu o n-gramy znakowe (2-4 znaki), omijające ograniczenia tradycyjnego podziału na słowa.
4. **Badane modele:** Implementacja i bezpośrednie starcie trzech klasyfikatorów: Naiwnego Bayesa, k-Nearest Neighbors (k-NN) oraz Random Forest.
5. **Kryteria sukcesu:** Optymalizacja pod kątem wysokiej precyzji (minimalizacja fałszywych alarmów – *False Positives*), kluczowej dla wdrożeń w środowiskach produkcyjnych typu WAF (*Web Application Firewall*).

Dlaczego logi HTTP są *trudne*?

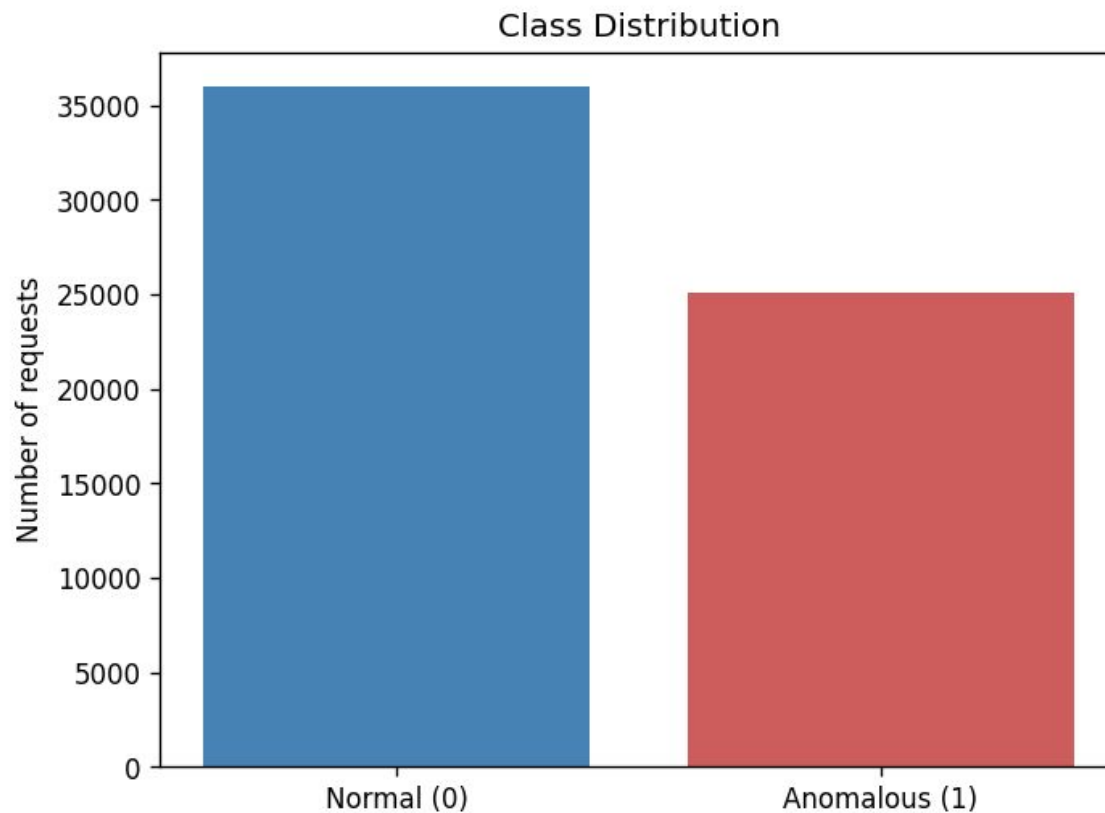
1. **Nieregularna struktura payloadów:** Kod złośliwy (*SQLi*, *XSS*) nie przypomina języka naturalnego. Brakuje w nim standardowych słów, składni czy spacji.
2. **Celowa obfuskacja:** Hakerzy omijają proste filtry stosując m.in. URL Encoding (np. %27 zamiast apostrofu '), wariacje wielkości liter lub wstrzykiwanie komentarzy w środek poleceń.
3. **Awaria tradycyjnej tokenizacji:** Standardowe algorytmy dzielące tekst na słowa po spacji są bezużyteczne. Złośliwy skrypt np. `<script>alert(1)</script>` dla klasycznego tokenizatora jest jednym, niezrozumiałym "słowem".
4. **Zależność od kontekstu:** Te same znaki specjalne (np. =, ., ?) w jednym zapytaniu są niezbędną częścią prawidłowego URL, a w innym – krytycznym elementem ataku Path Traversal (np. `../../../etc/passwd`).

Klasyfikacja Binarna i Charakterystyka Danych

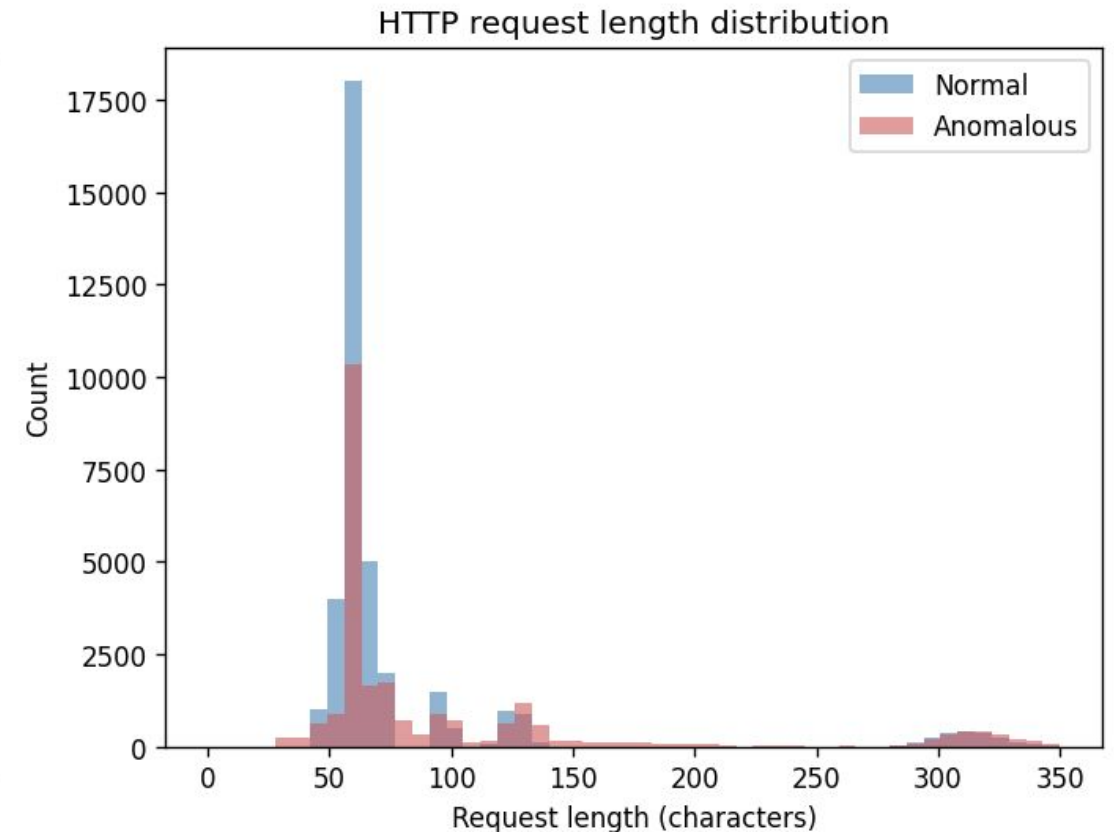
1. **Definicja problemu:** Uczenie nadzorowane (Supervised Learning) w modelu klasyfikacji binarnej. Każde żądanie HTTP to decyzja zero-jedynkowa: **0** (Ruch normalny) lub **1** (Anomalia/Atak).
2. **Zbiór danych (CSIC 2010):** Precyzyjnie zaetykietowany, syntetyczny ruch sieciowy, podzielony na ruch złośliwy (ok. 41%) oraz normalny (ok. 59%).
3. **Zakres cech wejściowych:** Analiza oparta wyłącznie na strukturze adresu URL oraz zawartości ciała żądania (POST body).
4. **Brak konieczności resamplingu:** Dzięki dobrej proporcji (41/59), modele uczą się stabilnie bez konieczności agresywnego dodawania sztucznych danych (co potwierdziły testy z użyciem narzędzia *RandomOverSampler*).

Problem rozpoznawania inputu

- 36000 normalnych zapytań vs 25065 złośliwych (59/41).
- Złośliwe zapytania są zwykle dłuższe.



Wykres 1: Dystrybucja klas datasetu



Wykres 2: Dystrybucja długości zapytań HTTP

Problem rozpoznawania inputu

Częstotliwość występowania znaków różni się między zapytaniami. Złośliwe zapytania zawierają bardziej nietypowe znaki (kodowanie URL, czy operatory SQL).



Wykres 3: Najczęściej występujące znaki w zapytaniach HTTP

Architektura i Pipeline Rozwiązania

1. **Modułowe podejście:** Rozbicie logiki na niezależne skrypty (*preprocess.py* oraz *train.py*), co umożliwia łatwe importowanie funkcji do późniejszych, zautomatyzowanych eksperymentów.
2. **Zastosowanie skryptów:**
 - a. *preprocess.py* - Oczyszczanie danych (URL decoding, normalizacja wielkości liter). Wektoryzacja TF-IDF z użyciem n-gramów znakowych (2-4).
 - b. *train.py* - Konfiguracja trzech modeli bazowych na tych samych cechach TF-IDF.
3. **Automatyzacja:** Moduły zwracają gotowe struktury (np. *get_splits()*, *get_models()*), eliminując powtarzanie kodu w fazie dalszych eksperymentów.

Architektura i Pipeline

Rozwiązania - Faza Preprocessingu.

1. **N-gramy znakowe:** Ekstrakcja zbitków (2 do 4 znaków) z zachowaniem granic słów, limitowana do 10 000 najważniejszych cech.
2. **Skalowanie *sublinear_tf*:** Logarytmiczne tłumienie powtarzających się znaków.
3. **Stratyfikacja:** Gwarancja zachowania naturalnej proporcji ataków (41%) w obu zbiorach po podziale 80/20.
4. **Ochrona przed wyciekami:** Ścisłe rozdzielenie budowy słownika (*fit_transform* na treningu) od samej translacji (*transform* na teście).

```
def build_vectorizer():
    return TfidfVectorizer(
        analyzer="char_wb",    # character n-grams, padded at word
        boundaries
        ngram_range=(2, 4),    # bigrams to 4-grams
        max_features=10_000,    # cap vocabulary size for speed
        sublinear_tf=True,      # apply 1+log(tf) scaling
    )

def get_splits(test_size=0.2, random_state=42):
    labels, texts = load_raw() # load data to analyze
    texts = clean(texts)       # URL-decode and lowercase

    vec = build_vectorizer()

    # Split on raw text first so the vectorizer is fit only on
    training data
    X_train_raw, X_test_raw, y_train, y_test = train_test_split(
        texts, labels, test_size=test_size,
        random_state=random_state, stratify=labels
    )
    X_train = vec.fit_transform(X_train_raw)
    X_test = vec.transform(X_test_raw)

    return X_train, X_test, y_train, y_test, vec
```

Term Frequency – Inverse Document Frequency.

TF (Term Frequency)	IDF (Inverse Document Frequency)
Częstotliwość terminu. Liczy, jak często dany n-gram (<i>np.</i> ' or) występuje wewnątrz tego jednego, konkretnego żądania HTTP.	Odwrotna częstotliwość dokumentu. Ocenia, jak rzadki i unikalny jest ten znak w skali całego zbioru logów. Skupia uwagę modelu na znakach nietypowych, a obniża wagę tych, które pojawiają się wszędzie.

Request (Wiersz)	Kolumna 1: <i>?id</i>	Kolumna 2: <i>=1</i>	Kolumna 3: <i>1'</i> (Atak)	Kolumna 4: <i>buty</i>
#1 (Krótki)	0.32 (2/6.16)	0.49 (3/6.16)	0.81 (5/6.16)	0.00
#2 (Długi)	0.17 (2/11.75)	0.26 (3/11.75)	0.43 (5/11.75)	0.85 (10/11.75)

Tabela 1: Przykład macierzy rzadkiej powstałej z połączenia wektorów

Architektura i Pipeline

Rozwiązania - Faza Trenowania.

1. **Trzy filary klasyfikacji:** Zestawienie algorytmów o odmiennym podejściu do danych: probabilistycznego (Naive Bayes), opartego na odległości (k-NN) oraz zespołowego (Random Forest).
2. **Optymalizacja dla macierzy rzadkich:** Zastosowanie metryki cosinusowej w algorytmie k-NN.
3. **Las Losowy:** Wykorzystanie 100 niezależnych drzew decyzyjnych w celu maksymalizacji stabilności i uniknięcia przeuczenia.
4. **Zautomatyzowana Ewaluacja:** Generyczna pętla przeprowadzająca proces uczenia oraz testowania, która natychmiast generuje pełny raport z metrykami *Precision*, *Recall* oraz *F1*.

```
def get_models():
    return {
        "Naive Bayes": MultinomialNB(alpha=0.1),
        "k-NN (k=5)": KNeighborsClassifier(n_neighbors=5,
metric="cosine", n_jobs=-1),
        "Random Forest": RandomForestClassifier(n_estimators=100,
n_jobs=-1, random_state=42),
    }

def train_and_evaluate(models, X_train, X_test, y_train, y_test):
    results = {}
    for name, clf in models.items():
        print(f"\n=== {name} ===")
        clf.fit(X_train, y_train)
        y_pred = clf.predict(X_test)
        report = classification_report(y_test, y_pred,
target_names=["Normal", "Anomalous"], output_dict=True)

        # Print human-readable summary
        print(classification_report(y_test, y_pred,
target_names=["Normal", "Anomalous"]))
        results[name] = {"clf": clf, "report": report}
    return results
```

Charakterystyka Wykorzystanych Algorytmów

Multinomial Naive Bayes (Podejście Probabilistyczne)	k-Nearest Neighbors (Podejście oparte na Podobieństwie):	Random Forest (Podejście Zespołowe / Ensemble)
Niezwykle szybki w działaniu	Analizuje 5 historycznych żądań o najbardziej zbliżonej strukturze do badanego logu ($k=5$).	Zbudowany ze 100 niezależnych drzew decyzyjnych, które wspólnie głosują nad ostateczną klasyfikacją.
Zakłada pełną niezależność cech (<i>naiwność</i>), co w naszym przypadku skutkowało wyższym wskaźnikiem fałszywych alarmów (niższe <i>Precision</i>).	Wykorzystuje odległość cosinusową, badając kąt między wektorami w przestrzeni 10 000 wymiarów, co uodparnia model na różnice w fizycznej długości zapytań HTTP.	Wybitnie odporny na szum w danych i przeuczenie (<i>Overfitting</i>). Osiągnął najwyższą precyzję , idealnie odnajdując nieliniowe wzorce w wygenerowanej macierzy rzadkiej.

Porównanie Klasyfikatorów (Walidacja Krzyżowa)

Eksperyment pierwszy

Model	Accuracy	Precision	Recall	F1
Naive Bayes	0.819	0.706	0.958	0.813
k-NN (k=5)	0.960	0.947	0.957	0.952
Random Forest	0.978	0.989	0.956	0.972

Tabela 2: Wyniki dla klasyfikatorów (walidacja krzyżowa)

1. **Metodologia oceny:** Zastosowanie 5-krotnej walidacji krzyżowej ze stratyfikacją.
2. **Ochrona przed wyciekiem danych:** Wektoryzator TF-IDF był trenowany dynamicznie, wyłącznie na podzbiorach treningowych w każdej z pięciu iteracji.
3. **Analiza wyników:**
 - Random Forest: Zwycięzca eksperymentu.
 - Naive Bayes: Wysoka czułość, ale drastycznie niska precyzja, co dyskwalifikuje go ze względu na zalew fałszywych alarmów (*False Positives*).

Wpływ Oversamplingu na skuteczność modeli

Eksperyment drugi

Model	Condition	Acc.	Prec.	Rec.	F1
Naive Bayes	baseline	0.823	0.711	0.958	0.816
Naive Bayes	oversam-pled	0.823	0.711	0.959	0.816
Random Forest	baseline	0.978	0.991	0.954	0.972
Random Forest	oversam-pled	0.976	0.987	0.954	0.970

*Tabela 3: Wyniki dla klasyfikatorów
(Oversampling)*

1. **Cel eksperymentu:** Weryfikacja, czy naturalne, łagodne niezbalansowanie zbioru negatywnie wpływa na proces decyzyjny klasyfikatorów.
2. **Metodologia:** Zastosowanie narzędzia RandomOverSampler w celu sztucznego wyrównania klas do proporcji 50/50 w fazie treningu.
3. **Wyniki:** Wpływ oversamplingu okazał się całkowicie marginalny.
4. **Decyzja inżynierska:** Ostateczne odrzucenie oversamplingu w architekturze produkcyjnej.

Analiza Statystyczna Wyników - Eksperyment 1

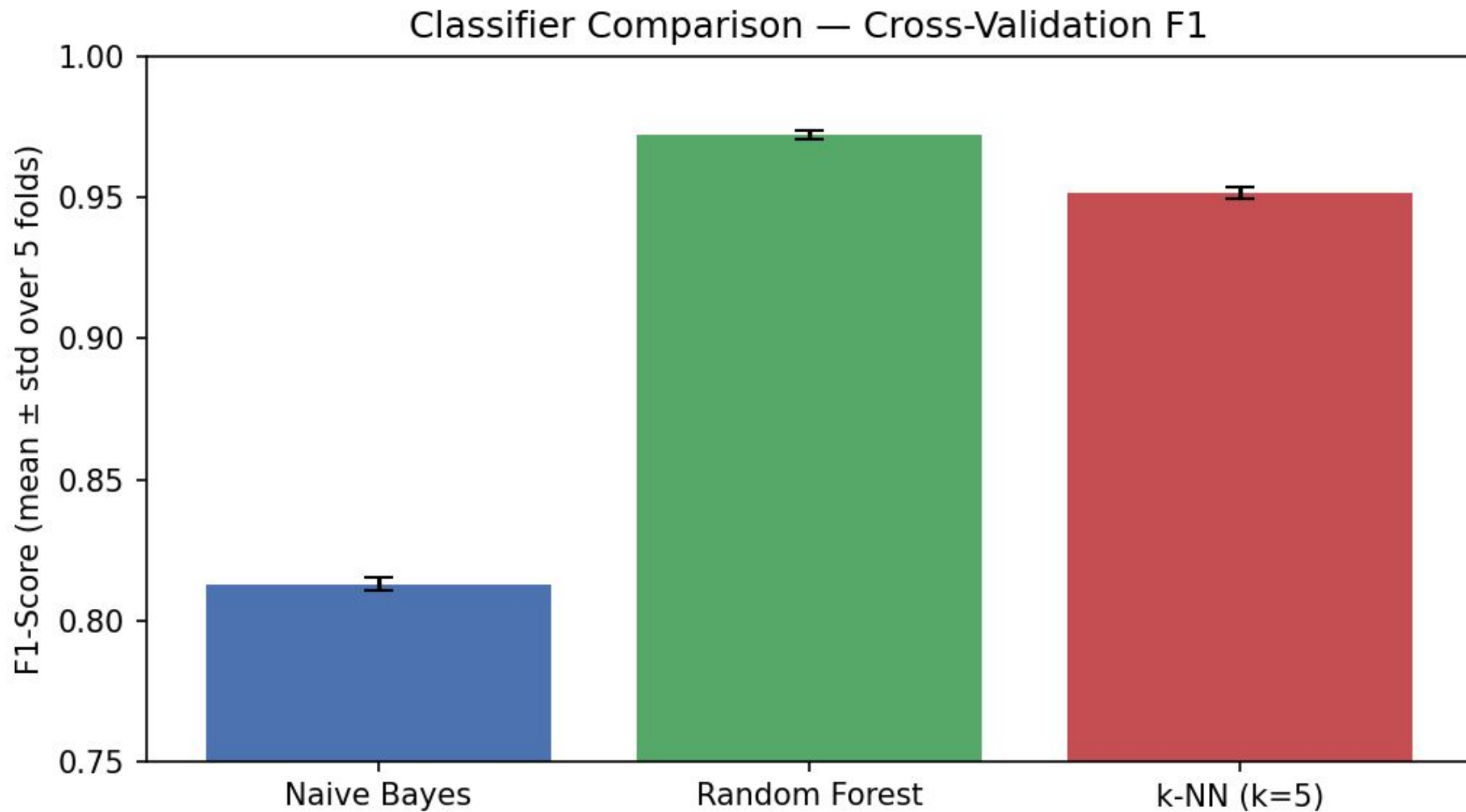
Model	Mean F1	Std
Naive Bayes	0.813	0.002
k-NN (k=5)	0.952	0.002
Random Forest	0.972	0.001

Tabela 4: Średnia metryka F1 oraz odchylenie standardowe dla 5-krotnej walidacji krzyżowej.

1. **Wysoka stabilność algorytmów:** Ekstremalnie niskie odchylenie standardowe ($Std \leq 0.002$) dowodzi, że modele nie są podatne na losowość i osiągają powtarzalne wyniki niezależnie od podziału danych.
2. **Test Friedmana:** Potwierdził globalną, statystyczną istotność różnic między metrykami F1 wszystkich trzech klasyfikatorów ($p < 0.05$).
3. **Test Wilcoxona (*k-NN* vs *Random Forest*):** Bezpośrednie starcie dwóch najlepszych modeli dało wynik $p=0.0625$. Przekroczenie rygorystycznego progu jest tu wyłącznie anomalią matematyczną wynikającą z małej liczby iteracji ($n=5$).



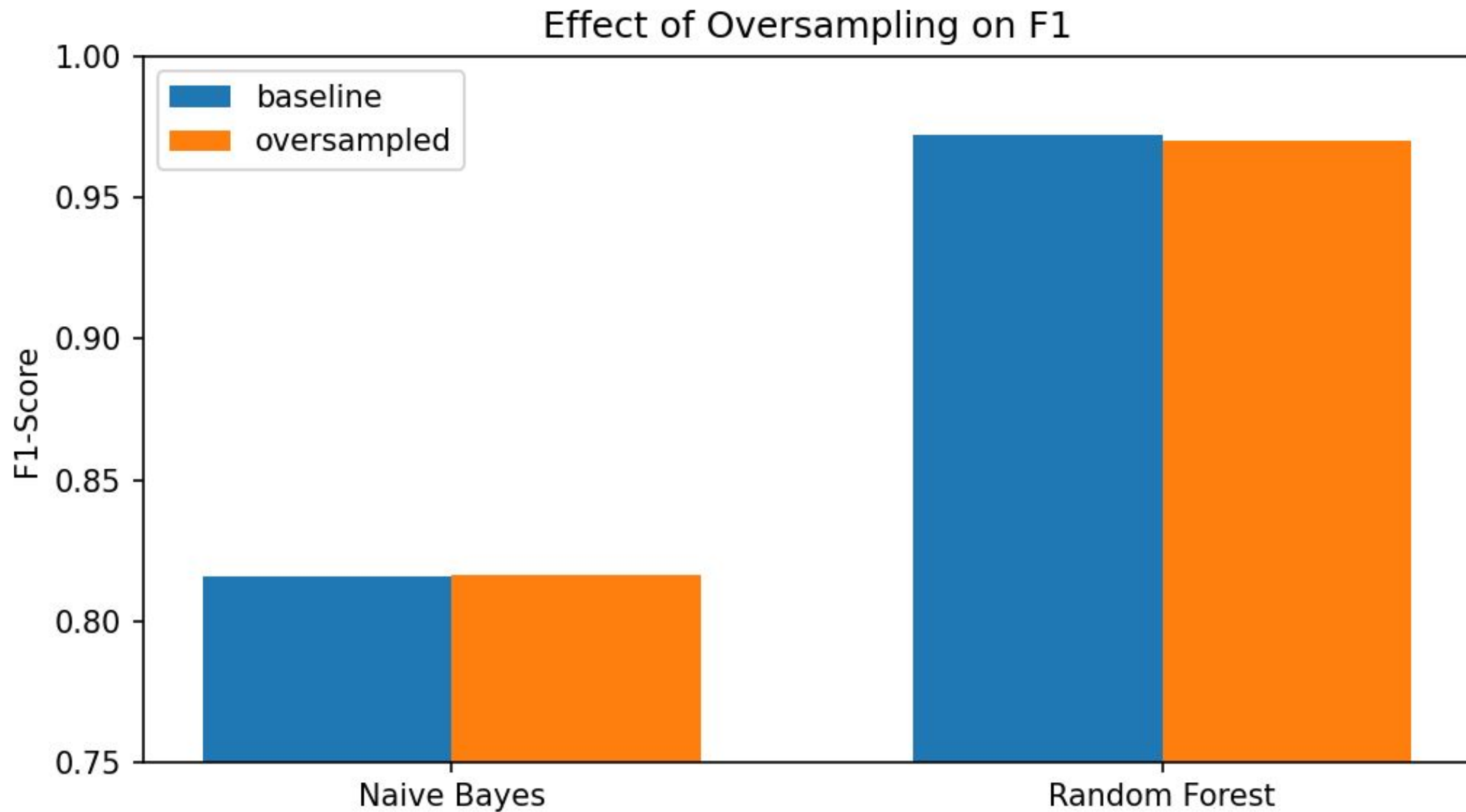
Politechnika
Wrocławska



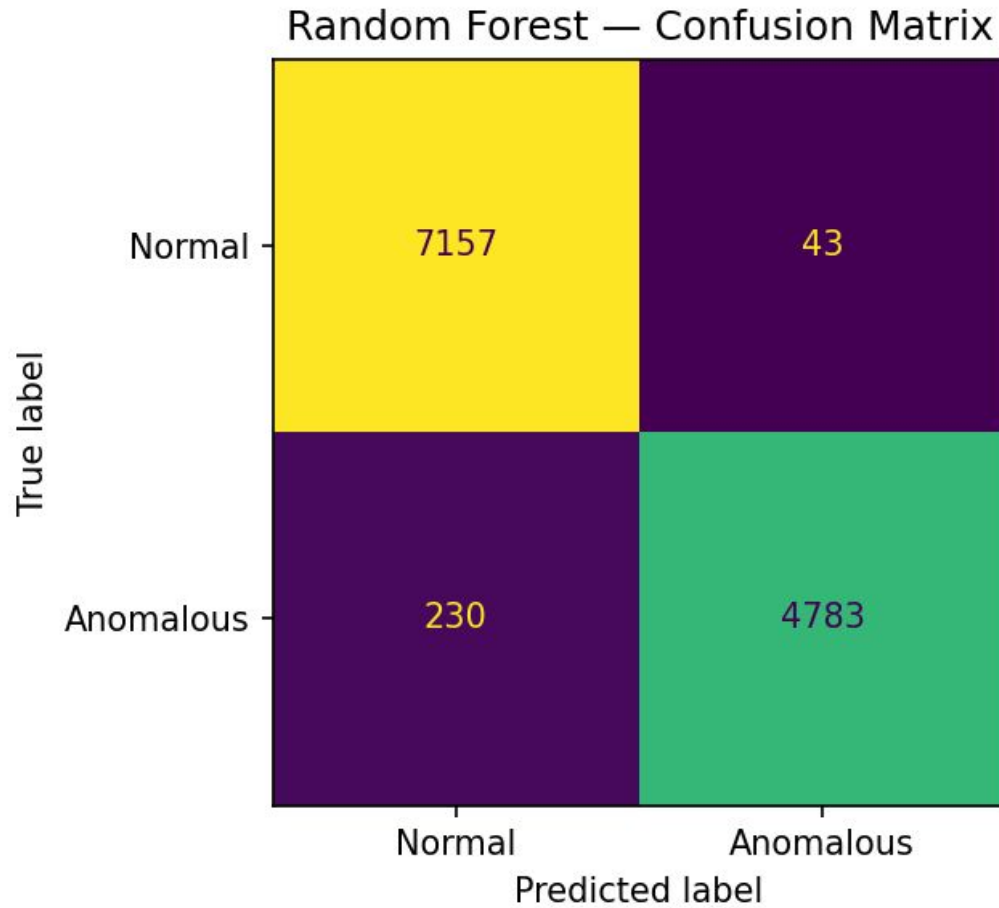
Wykres 4: Wizualizacja wyników (metryka F1) z słupkami błędów



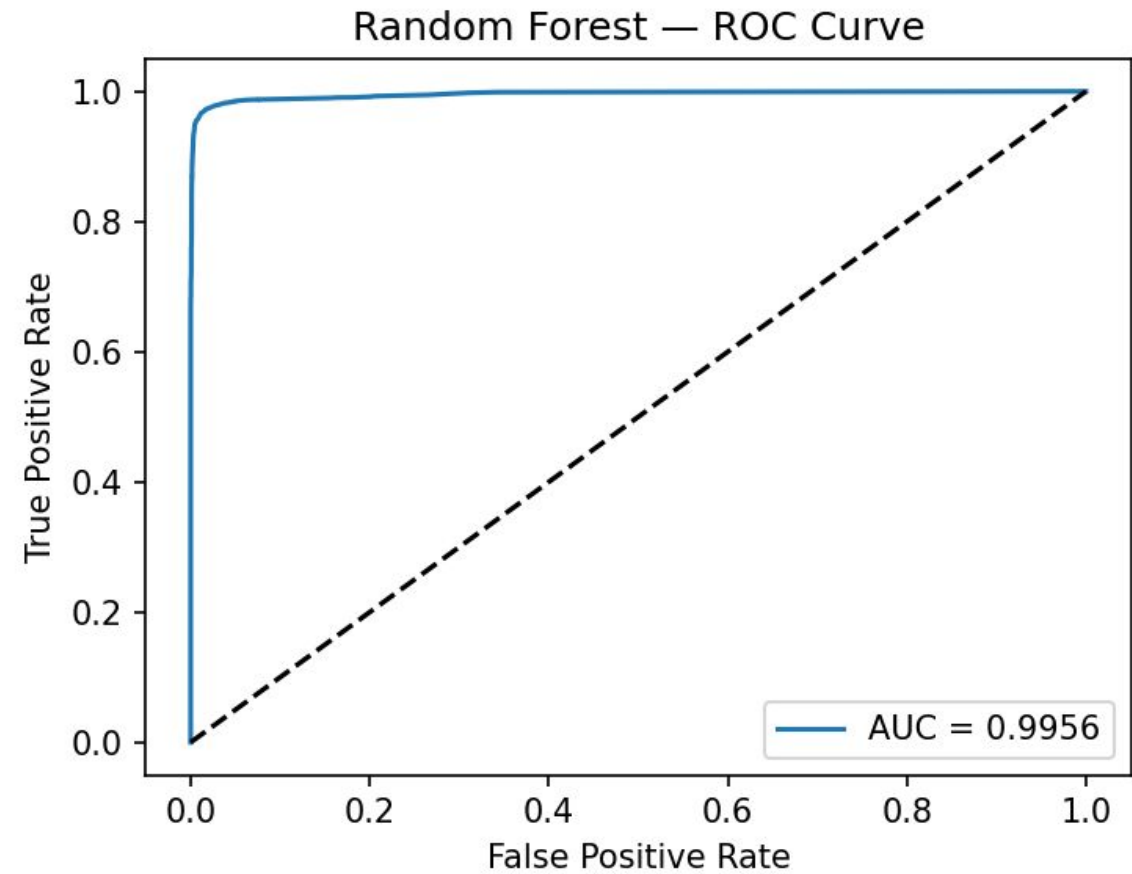
Politechnika
Wrocławska



Wykres 5: Wpływ oversamplingu na wyniki F1 klasyfikatorów



Wykres 6: Random Forest - macierz błędów



Wykres 7: Random Forest - krzywa ROC

Wnioski Końcowe (Dlaczego to działa?)

1. **Zwycięska architektura:** Zestawienie n-gramów znakowych (*TF-IDF*) z modelem Random Forest to optymalne rozwiązanie dla systemów WAF.
2. **Siła n-gramów:** Analiza zbitków 2-4 znaków omija problem tokenizacji słów i bezpośrednio uderza w składnię używaną przez hakerów.
3. **Brak konieczności oversamplingu:** Modele zespołowe udowodniły odporność na łagodne niezbalansowanie klas (41/59), czyniąc sztuczne powielanie danych zbędnym narzutem obliczeniowym.

Ograniczenia Projektu (Kierunki Rozwoju)

1. **Wiek zbioru danych (CSIC 2010):** Brak reprezentacji nowoczesnych wektorów ataków (np. *SSRF*, *XXE*, *GraphQL Injection*).
2. **Specyfika jednej aplikacji:** Ryzyko przeuczenia (overfittingu) pod konkretną strukturę adresów URL syntetycznej bazy zapytań HTTP.
3. **Brak ewaluacji czasowej:** Ruch sieciowy ewoluuje. Walidacja krzyżowa tasowała logi losowo, zamiast uczyć model na przeszłości i testować na przyszłości.
4. **Wąski zakres cech:** Analiza objęła tylko URL i ciało POST. Zignorowano nagłówki HTTP (*User-Agent*, *Cookie*, *Referer*), które niosą ogromną wartość analityczną.



Politechnika
Wrocławska

Dziękuję za uwagę

unite!
University Network for Innovation,
Technology and Engineering



HR EXCELLENCE IN RESEARCH